Parallel Implementation of the Lattice Boltzmann Method: 2D flow past a cylinder

June 2026

Ghassan Abboud

I. Introduction

The lattice Boltzmann method (LBM) is a computational fluid dynamics approach that models fluid flow through the evolution of particle distribution functions f_i on a discrete lattice. It replaces the continuum viewpoint of directly solving the Navier-Stokes equations with a mesoscopic perspective that naturally captures local particle interactions, complex boundaries, and porous media [1]. It has been applied in many engineering areas including aerodynamics, microfluidics and multiphase flow [2]. LBM is derived from the Boltzmann equation:

$$\frac{\partial f}{\partial t} + \boldsymbol{c} \cdot \nabla f = \Omega \tag{1}$$

where $f(\boldsymbol{x}, \boldsymbol{c}, t)$ represents the probability density function (PDF) of particles with a specific position $\boldsymbol{x}$ and velocity $\boldsymbol{c}$. Velocity space is discretized into a finite set of lattice directions f_i forming a stencil. This project focuses on the D2Q9 stencil, which introduces nine lattice directions.

LBM updates each time step through two local operations. During collision, cell populations relaxed toward equilibrium according to the collision operator Ω, a purely local operation. During streaming, the post-collision populations are propagated to neighboring cells along their discrete velocities.

Beyond its modeling flexibility, LBM is interesting from a computational point of view because its operations are highly local and well suited to parallel execution [3]. Concurrently, performance is often limited by memory bandwidth rather than arithmetic cost, since the streaming step requires frequent data movement between neighboring cells [3].

II. Methodology

A. Memory access patterns

Lattice nodes are stored as one array of dimension $N_x \cdot N_y \cdot Q$. The choice of index order has an impact on cache reusability, yielding different advantages for different architectures. In a Structure-of-Arrays (SoA) layout, the slowest moving index corresponds to the velocity population i, meaning values for f_i of adjacent cells are stored contiguously for any i. In an Array-of-Structures (AoS) layout, the fastest moving index corresponds to i, meaning all population values f_0 to f_9 are contiguous for any given cell.

B. One-step algorithm

The provided algorithm used a naïve two-step approach, with two separate loops for collision and streaming. This increased the amount of data moved for one lattice update (LU). Before tackling parallelization, a one-step algorithm combined collision and streaming in one loop. Both SoA and AoS versions were implemented and using pull scheme, as it has been shown to reduce memory bandwidth by replacing uncoalesced writes with uncoalesced reads [4]. Single-CPU performance was calculated on a 800×400 domain using 8 replicates.

C. MPI parallelization

Two parallel implementations of the main collide-stream kernel were developed. The first decomposes the simulation domain along rows, such that each rank has two neighboring ranks: one above and one below. The second uses the `MPI_Cart` routines to decompose the domain along both spatial dimensions.

Ghost exchanges are performed before each time step. In the `MPI_Cart` implementation, special care is taken to ensure that corner ghost cells are correctly propagated. Both implementations are validated through manual inspection of streaming at selected locations in the domain, and by verifying a Strouhal number of 0.16 on a 800×400 domain.

D. CUDA implementation

Two CUDA implementations were developed: one using the SoA layout and one using the AoS layout. In both cases, small constant variables such as the stencil offsets and weights were loaded into constant device memory. The `solid` array, which identifies wall cells, was stored in the texture cache. The two main arrays, `f_` and `ftmp_`, were stored in global memory. Three CUDA kernels were implemented: one for the fused collide-stream operation and one each for inlet and outlet treatment. In all kernels, each GPU thread handles one LU per step. Array indexing ensures coalesced global memory accesses in the SoA version. Block dimensions are specified as CLI arguments, while grid size is set to the minimum needed to accommodate the entire domain.

III. Results & Discussion

A. Single-CPU performance

The one-step implementation reduced the memory requirement from 360 B/LU to 216 B/LU, thereby increasing the algorithm's arithmetic intensity (AI) and shifting it closer to a compute-bound regime on the CPU. This change translated into improved performance, increasing from 21.46 ± 1.37 MLUPS for the two-step implementation to 27.07 ± 1.59 MLUPS for the AoS one-step version and 22.70 ± 0.92 MLUPS for the SoA version (Fig. 1). The superior performance of the AoS layout is consistent with profiling results, which identified the collision step as the dominant kernel, accounting for 52.11% of the total runtime compared to 44% for streaming. As AoS is collision-optimized, it enables cache reuse when accessing all PDFs of a single cell for collision. The SoA version was nonetheless selected for parallelization due to its use in GPU parallelization literature [4].

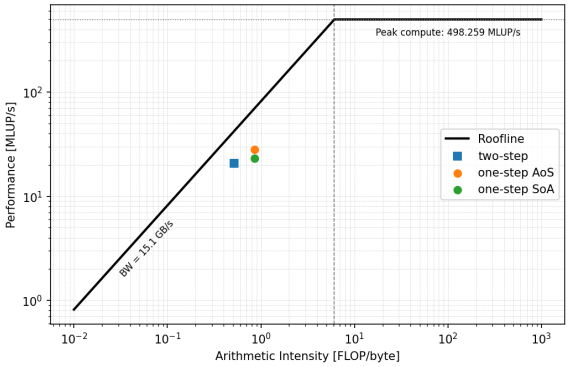


Figure 1: Roofline model for the single-CPU implementation comparing performance in MLUPS to arithmetic intensity on a logarithmic scale. Hardware memory bandwidth (BW) and peak performance were measured with helper programs `STREAM` and `dgemm`. Three points represent the original two-step implementation provided and a one-step implementation with both AoS and SoA memory layouts.

B. Strong Scaling

Strong scaling analysis revealed impressive performance of the `MPI_Cart` implementation, closely matching Amdahl's law for maximum theoretical speedup up until 64 (Fig. 2). In contrast, the row-wise implementation showed satisfactory speedup up until 8 cores. Its performance then degraded: it achieved a speedup of 18.70 on 64 cores, compared to `MPI_Cart`'s speedup of 49.47.

The row-wise implementation introduces significant communication overhead at high ranks. Indeed, the implementation's communication-to-computation workload is given by:

$$\text{ratio} = \frac{O(2n_x)}{O(n_x \cdot n_{y,\text{local}})} = O(\frac{1}{n_{y,\text{local}}}) = O(\frac{p}{N_y})$$

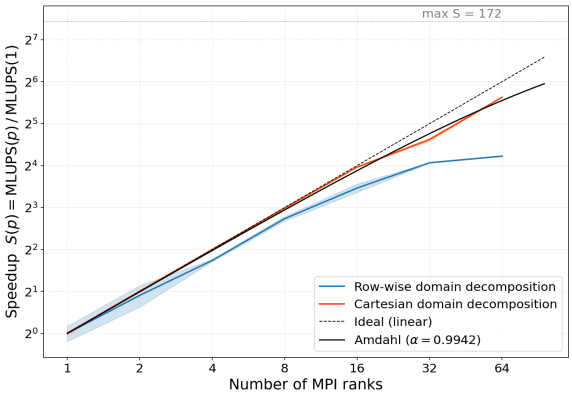


Figure 2: Strong scaling analysis showing execution speedup $S(p) = \frac{T_1}{T_p}$ for both row-wise and `MPI_Cart` implementation on a 1600×800 grid for 60000 steps. An inlet velocity of 0.05 and a Reynolds number of 100 were used. Amdahl's law, showing maximal achievable speedup is plotted with α derived from profiling the serial one-step kernel using `perf`. Error bars show standard deviation over three replicates.

with p the number of processors, N_y the total number of rows and $n_{y,\text{local}}$ the number of rows assigned to each process. In contrast, for the `MPI_Cart` implementation, assuming a square rank grid so that $n_{x,\text{local}} = \frac{N_x}{\sqrt{p}}$ and $n_{y,\text{local}} = \frac{N_y}{\sqrt{p}}$:

$$\begin{aligned}\text{ratio} &= \frac{O(2n_{x,\text{local}} + 2n_{y,\text{local}})}{O(n_{x,\text{local}} \cdot n_{y,\text{local}})} \\ &= \frac{O(\frac{2}{\sqrt{p}}(N_x + N_y))}{O(\frac{N_x \cdot N_y}{p})} \\ &= O(\sqrt{p}(\frac{1}{N_x} + \frac{1}{N_y}))\end{aligned}$$

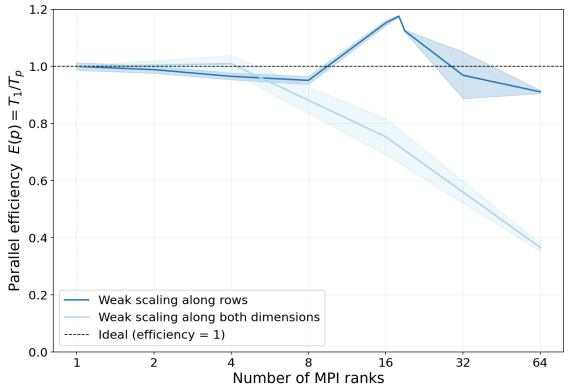
This better asymptotic communication-to-computation ratio of the `MPI_Cart` version explains the observed scaling gap. However, the current analysis is limited to 64 cores and further scaling is expected to reveal communication overhead in the `MPI_Cart` version as well.

C. Weak Scaling

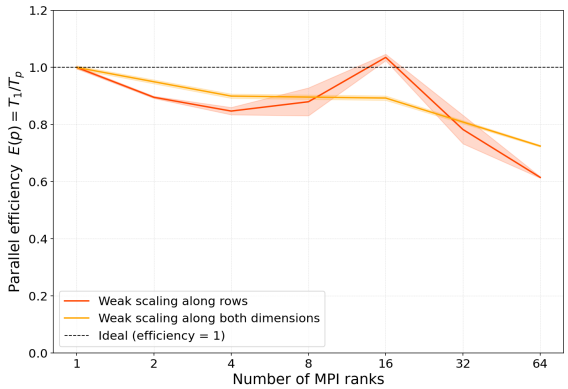
The analysis of weak scaling results along two test modalities reveals the failure modes of the row-based implementation. When increasing the number of rows only, the row-wise implementation retains high efficiency, reaching 91.1% at 64 cores (Fig. 3a). However, refining the grid along both dimensions, a more realistic scenario, leads to poor efficiency, as low as 36.6% on 64 cores (Fig. 3a). As both dimensions are scaled, the row-wise implementation assigns a smaller number of longer rows to each rank, increasing the computation-to-communication ratio. This observation initially motivated the development of the `MPI_Cart` method.

The `MPI_Cart` method better accommodates scaling of the grid in both dimensions, retaining 72.5% efficiency at 64 cores (Fig. 3b). The remaining gap is explained by communication overhead. `MPI_Cart` performs worse than the row-wise method when scaling along rows only, retaining only 61.5% efficiency at 64 cores (Fig. 3b). This indicates that additional optimizations could adapt the rank grid created by `MPI_Dims_create` to the size of the computational domain. This is left for future work.

Another interesting observation is the superlinear peak in performance when scaling along rows for both implementations at 16 ranks. Initial investigations were not able to determine its cause.



(a) Row-wise implementation



(b) `MPI_Cart` Implementation

Figure 3: Weak scaling analysis showing execution efficiency $E(p) = \frac{T_1}{T_p}$ for both row-wise and `MPI_Cart` implementations. For each implementation, two analyses are performed. The first only scales the number of rows from 200 to 12800 while the x dimensions is fixed at 400. The second scales both dimensions, from a 400×200 grid on one core to 3200×1600 on 64 cores. Error bars show standard deviation over three replicates.

D. CUDA performance

For both the SoA and AoS versions, performance increased with problem size, rising from 364.48 and 344.83 MLUPS, respectively, at 2^{13} lattice points to 4939.83 and 2851.07 MLUPS at 2^{25} lattice points (Fig. 4). SoA consistently outperformed AoS across all sizes and reached 79% of the theoretical peak performance on V100, suggesting a well-optimized implementation. This advantage comes from memory access patterns: in the SoA version, consecutive threads read consecutive locations in `f_` and `ftmp_`, enabling coalesced global memory transactions. By contrast, the AoS layout induces consecutive 9-strided accesses for D2Q9, which limits memory utilization to $\frac{8}{32} = 25\%$. The growth in performance with problem size likely reflects SM saturation, since enough blocks must be launched to keep the V100's 80 SMs occupied; this threshold is only reached from 163.8K lattice points onward, after which performance begins to plateau (Fig. 4).

For a fixed problem size with the SoA version, performance rose sharply as the block x dimension increased from 8 to 32 (Fig. 5). Beyond 32, the gains were more modest, but remained noticeably different between multiples and non-multiples of 32. Because CUDA blocks are executed in warps of 32 threads, non-multiples leave a trailing warp with idle threads. For block sizes below 32, performance drops rapidly because each warp uses only half or a quarter of its threads, which also reduces memory-access efficiency. Initial investigations did not reveal any dependence of performance on block y dimension.

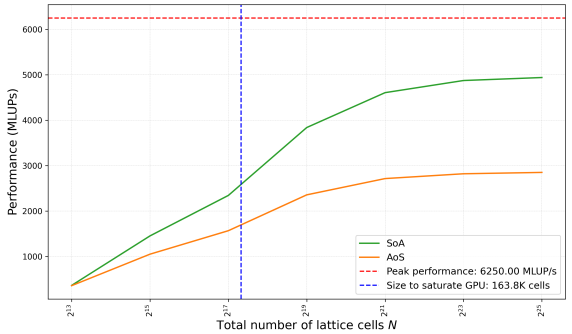


Figure 4: Dependence of raw performance in MLUPS on problem size measured as the total number of lattice cells N for both SoA and AoS implementations. Block dimensions were fixed to $(256, 1)$. The red dashed line shows the maximum theoretical memory-bound performance on a V100 assuming a maximum global memory bandwidth of 900 GB/s and an arithmetic intensity of 1.28.

IV. Conclusion

In conclusion, this study set out to parallelize the Lattice Boltzmann Method on both CPU clusters using MPI and single GPUs using CUDA. The scaling and performance analyses highlighted the close interplay between algorithm design and hardware characteristics, which is crucial for achieving good performance. One important lesson is that an optimization that works well on one architecture does not necessarily carry over to another. For instance,

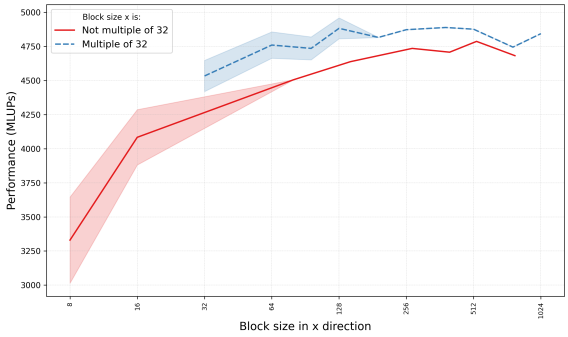


Figure 5: Dependence of SoA raw performance in MLUPS on block x dimension for a fixed problem size of 4092×2048. Block y dimension was fixed to 1. Results are segregated between multiples and non-multiples of 32.

the AoS layout performed better on the CPU because it was more favorable for collision handling, but it was a poor choice on the GPU, where coalesced global memory accesses are essential. The CUDA implementation delivered strong results and reached 79% of peak theoretical performance. The `MPI_Cart` implementation showed optimal strong scaling until 64 cores and better weak scaling for real-world scenarios but can still be improved, especially by overlapping communication with computation to hide ghost exchange latency.

V. AI USAGE STATEMENT

The author declares using the Claude Code VS Code extension to develop a substantial part of the code, but only for implementation. The parallelization strategy, scaling and performance experiments, interpretation of results, and subsequent design choices were all planned and decided by the author based on the results and the literature review.

Claude Code's output was checked in three ways:

1) All AI-generated code was read and annotated by the author to ensure understanding. The author therefore takes full responsibility for all code developed in this project.
2) Diagnostic prints were added at key points in the simulation domain to track population values over successive time steps. This was especially useful for the MPI version, where Claude Code had difficulty producing correct streaming behavior with halo data.
3) All implementations were validated on a 800×400 domain over 60,000 steps, yielding a Strouhal number of 0.16.

The author also declares using Perplexity AI to tighten paragraph phrasing and check grammar. However, no paragraph was generated from scratch.

REFERENCES

[1] D. Arumuga Perumal and A. K. Dass, "A Review on the development of lattice Boltzmann computation of macro fluid flows and heat transfer," *Alexandria Engineering Journal*, vol. 54, no. 4, pp. 955–971, Dec. 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1110016815001362

[2] K. V. Sharma, R. Straka, and F. W. Tavares, "Current status of Lattice Boltzmann Methods applied to aerodynamic, aeroacoustic, and thermal flows," *Progress in Aerospace Sciences*, vol. 115, p. 100616, May 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0376042120300282

[3] C. Körner, T. Pohl, U. Rüde, N. Thürey, and T. Zeiser, "Parallel Lattice Boltzmann Methods for CFD Applications," in *Numerical Solution of Partial Differential Equations on Parallel Computers*, A. M. Bruaset and A. Tveito, Eds. Berlin, Heidelberg: Springer, 2006, pp. 439–466.

[4] N.-P. Tran, M. Lee, and S. Hong, "Performance Optimization of 3D Lattice Boltzmann Flow Solver on a GPU," *Scientific Programming*, vol. 2017, pp. 1–16, Jan. 2017.

SUPPLEMENTARY DATA

Supplementary Table I: Summary of performance in MLUPS for different implementations and experiments.

Implementation	Problem Size	Performance in MLUPS (mean ± std)	Main Figure
serial - two-step	800×400	21.46 ± 1.37	Fig. 1
serial - one-step SoA	800×400	22.70 ± 0.92	Fig. 1
serial - one-step AoS	800×400	27.07 ± 1.59	Fig. 1
MPI - row-wise - 64 ranks	1600×800	367.97 ± 2.04	Fig. 2
MPI - cartesian - 64 ranks	1600×800	997.95 ± 1.08	Fig. 2
MPI - row-wise - 64 ranks	400×12800	967.04 ± 6.22	Fig. 3a
MPI - cartesian - 64 ranks	400×12800	833.03 ± 3.28	Fig. 3a
MPI - row-wise - 64 ranks	3200×1600	455.56 ± 17.13	Fig. 3b
MPI - cartesian - 64 ranks	3200×1600	983.82 ± 3.28	Fig. 3b
CUDA - SoA - block dimensions $(256, 1)$	8192×4096	4939.83 ± 0.20	Fig. 4
CUDA - AoS - block dimensions $(256, 1)$	8192×4096	2851.07 ± 0.20	Fig. 4